

Files

main

+

Q

Go to file

> .vscode

> __pycache__

.DS_Store

Metadata and Protein Data for Mo...

README.md

bar_graph.png

main.py

patient.py

scatter_plot.png

BME2315_Module01 / patient.py

LiamSunshine

Module 1 Coding Homework Complete

941a9b8 · 8 minutes ago

History

Code

Blame

77 lines (64 loc) · 2.54 KB

Raw

```
1  import csv
2
3  # Create a Patient class to store patient demographic and protein data
4  class Patient:
5      all_patients = []
6
7      # Initialize each patient with their demographic, clinical, and protein attributes
8      def __init__(self, donor_id: str, age_at_death: float, sex: str,
9                  cognitive_status: str, apoe_genotype: str, thal: str,
10                 braak: str, abeta40: float, abeta42: float,
11                 ttau: float, ptau: float):
12
13         self.donor_id = donor_id
14         self.age_at_death = age_at_death
15         self.sex = sex
16         self.cognitive_status = cognitive_status
17         self.apoe_genotype = apoe_genotype
18         self.thal = thal
19         self.braak = braak
20         self.abeta40 = abeta40
21         self.abeta42 = abeta42
22         self.ttau = ttau
23         self.ptau = ptau
24
25         Patient.all_patients.append(self)
26
27     # Represent each patient with identifying and demographic information
28     def __repr__(self):
29         return f"{self.donor_id}: {self.sex}, Age {self.age_at_death}, {self.cognitive_status}"
30
31     # Create Patient objects from each row of the CSV file
32     @classmethod
33     def instantiate_from_csv(cls, filename):
34         with open(filename, encoding="utf8") as f:
35             reader = csv.DictReader(f)
36
37             for row in reader:
38                 cls(
39                     donor_id=row["Donor ID"],
40                     age_at_death=float(row["Age at Death"]),
41                     sex=row["Sex"],
42                     cognitive_status=row["Cognitive Status"],
43                     apoe_genotype=row["APOE Genotype"],
44                     thal=row["Thal"],
45                     braak=row["Braak"],
46                     abeta40=float(row["ABeta40 pg/ug"]),
47                     abeta42=float(row["ABeta42 pg/ug"]),
48                     ttau=float(row["tTAU pg/ug"]),
49                     ptau=float(row["pTAU pg/ug"])
50                 )
51
52     # Filter patients based on selected attributes
53     @classmethod
54     def filter(cls, list, sex: str = "any", cognitive_status: str = "any"):
55         all_patients = list
56         remove_list = []
57
58         attr_list = (
59             sex,
60             cognitive_status
61         )
62
63         attr_name = (
64             "sex",
65             "cognitive_status"
66         )
67
68         for attr in range(len(attr_list)):
69             if attr_list[attr] != "any":
70                 for patient in all_patients:
71                     if getattr(patient, attr_name[attr]) != attr_list[attr]:
72                         remove_list.append(patient)
73
74         all_patients = [patient for patient in all_patients if patient not in remove_list]
75         remove_list.clear()
76
77         return all_patients
```